ROS 2: Concepts and Implementation

Sree Geethika Maduri
Driverless-Taskphase
(of Affiliation)
Formula Manipal
(of Affiliation)
Manipal,India
sree.mitmpl2026@learner.manipal.edu

***Abstract*—This report includes the basic concepts of ROS 2, including nodes, topics, services, actions, parameters, and workspaces. It also covers the two tasks: calculating longitudinal speed of a wheel using angular velocity and radius, and checking whether a given string is a palindrome using multiple nodes.**

Keywords—ROS 2, Nodes, Topics, Services, Actions, Parameters, Colcon, QoS.

I. Introduction

Robot Operating System 2 (ROS 2) is an open-source framework used to develop software for robots and other distributed applications. It provides libraries, tools, and communication mechanisms that allow different programs to exchange information and work together. ROS 2 applications are divided into smaller independent components called nodes. These nodes communicate using topics, services, and actions, depending on the requirements of the application. This modular approach makes robotic software easier to develop, test, and maintain. ROS 2 also provides tools such as colcon for building packages and rqt_graph for visualizing communication between nodes. The objective of this task is to understand these concepts and implement two applications: a wheel longitudinal speed calculator and a three-node palindrome checker.

II. ROS 2 Computation Graph

The ROS 2 computation graph represents the nodes in a system and the communication connections between them. It helps developers understand how information flows through an application. The main components of the computation graph are nodes, topics, services, and actions. Nodes perform specific tasks, while topics, services, and actions provide different methods of communication. Messages define the data exchanged between nodes. For example, in a wheel-speed calculation application, one node publishes angular velocity and wheel radius, while another node receives these values and calculates the longitudinal speed.

III. Concepts Of ROS 2

A. Nodes

A node is an independent executable that performs a specific task in a ROS 2 application. An application can contain multiple nodes that communicate to complete a larger task. For example, the palindrome checker uses separate nodes to publish a string, check whether it is a palindrome, and generate a response.

B. Topics

Topics are named communication channels that allow nodes to exchange messages using a publish-subscribe model. A publisher sends messages to a topic, and subscribers receive those messages. This communication is suitable for continuous data, such as sensor readings, wheel measurements, and status updates.

C. Services

Services use a request-response model. A client sends a request to a service, which processes it and returns a response. Services are useful for specific operations, such as resetting a value or requesting a configuration change, where a result is expected.

D. Actions

Actions are designed for operations that take time to complete. They support goal submission, progress feedback, and a final result. For example, a robot navigating to a destination can use an action to report its progress while moving.

E. Parameters

Parameters are configurable values associated with a node. They allow developers to change settings without modifying the source code. Examples include speed limits, thresholds, and publishing frequencies.

F. Logging and Launching Nodes

ROS 2 provides logging tools to display informational messages, warnings, and errors during execution. The rqt_console tool helps developers view and filter log messages. Launch files allow multiple nodes and their configurations to be started together, simplifying the execution of applications containing several nodes.

G. Difference Between Topics and Services

Topics use a publish-subscribe model and are suitable for continuous or repeated data exchange. Services use a request-response model and are suitable when a client requests a specific operation and expects a response. For example, wheel measurements can be published through a topic, whereas requesting a node to reset a value is better handled by a service.

IV. Custom Interfaces

ROS 2 interfaces define the structure of data exchanged between nodes. They include messages, services, and actions. Custom interfaces can be created when the standard message types do not contain the fields required by an application. A custom message specifies the fields and data types that a message contains. In this task, TaskOne stores wheel angular velocity and radius, while RT2String stores the input string. These messages are defined in the

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

taskone_interfaces package and used by the nodes for communication.

V. Workspaces, Package and Colcon

A workspace contains the packages required for a ROS 2 project. The workspace used here is ros2_ws, with source packages stored in the src directory. The build, install, and log directories store build files, installed packages, and build logs, respectively. The project uses taskone_interfaces for custom messages and taskone_nodes for Python nodes. Colcon builds these packages and their dependencies using the colcon build command.

VI. Quality Of Service(QoS)

Quality of Service (QoS) controls how messages are delivered between ROS 2 nodes. Its main policies include reliability, durability, history, and depth. In this project, publishers and subscribers use a queue depth of 10, while other QoS policies follow ROS 2 defaults. Appropriate QoS settings help ensure communication suits the application's requirements.

VII. Implementation Of Task 1

A. Objective

The objective is to calculate the longitudinal speed of a wheel using its angular velocity and radius. The application uses two ROS 2 nodes that communicate through a custom message.

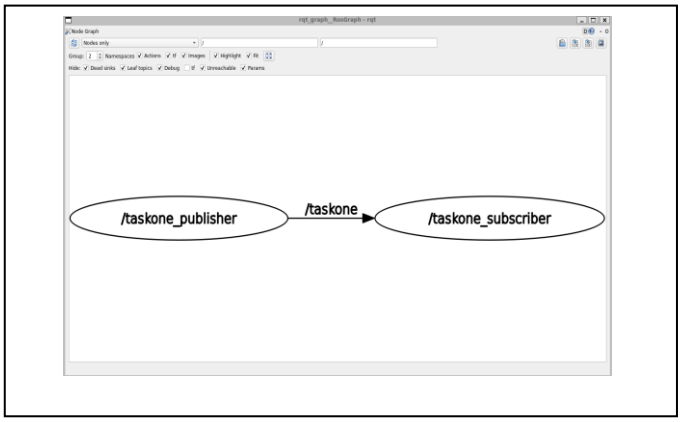
B. Implementation

Node 1 accepts the wheel's angular velocity and radius as input and publishes them to the /taskone topic using the custom TaskOne message. The message contains the fields ang_vel and radius, both defined as float32. Node 2 subscribes to /taskone and calculates the longitudinal speed using the equation:

$$v = \text{angular velocity} * \text{radius}$$

C. Result

For an angular velocity of 4 rad/s and a wheel radius of 0.2 m, the calculated speed is 0.8 m/s. The ROS 2 graph confirms communication between the publisher and subscriber nodes.



VIII. Implementation Of Task 2

A. Objective

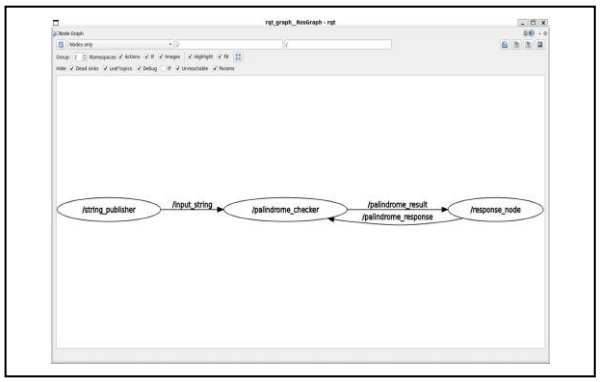
The objective is to determine whether an input string is a palindrome using three communicating ROS 2 nodes. The nodes exchange data through topics to check the string and return a "Yes" or "No" response.

B. Nodes

Node 1, string_publisher, accepts a string and publishes it to the /input_string topic using the custom RT2String message. Node 2, palindrome_checker, subscribes to /input_string and checks whether the string is equal to its reverse. It publishes the result as an integer on /palindrome_result, where 1 indicates a palindrome and 0 indicates a non-palindrome. Node 3, response_node, receives the result and publishes a "Yes" or "No" response on /palindrome_response. Node 2 subscribes to this topic and displays the response.

C. Result

The application was tested using the strings madam and hello. The string madam was identified as a palindrome and returned “Yes”, while hello was identified as a non-palindrome and returned “No”. The ROS 2 computation graph confirmed successful communication between all three nodes.



IX. Conclusion

This task provided practical experience with ROS 2 nodes, topics, custom messages, and communication mechanisms. The wheel speed calculator demonstrated data exchange between a publisher and subscriber, while the palindrome checker demonstrated communication among three nodes. Both implementations helped develop an understanding of ROS 2 package building, node execution, and computation graph visualization.

References

[1] ROS 2 Documentation, “Ubuntu Install (Debian Packages).” [Online]. Available: https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html

[2] ROS 2 Documentation, “Beginner: CLI Tools.” [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html

[3] GeeksforGeeks, “How to Set Up Dual Boot with Linux and Windows.” [Online]. Available: https://www.geeksforgeeks.org/techtips/setup-dual-boot-with-linux-and-windows